

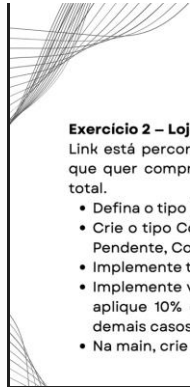
EXERCÍCIO 1

Exercício 1 – Café Leblanc

No Café Leblanc, Sojiro anota cada pedido feito durante o dia. Cada bebida tem um nome, um tipo e um preço. Ao final, ele precisa saber o valor total do pedido.

- Defina o tipo `Bebida` com os campos `nome`, `tipo` e `preco`.
- Crie um tipo `StatusPedido` com os construtores `Aberto`, `Entregue` e `Cancelado`.
- Crie o tipo `Pedido` com uma lista de bebidas e um `StatusPedido`.
- Implemente `valorTotalPedido :: Pedido -> Double` que soma os preços das bebidas. Use guards para retornar `0.0` se o pedido estiver `Cancelado`, e o total simples nos demais casos.
- Implemente `primeiraBebida :: Pedido -> String` usando pattern matching na lista de bebidas – retorne o nome da primeira ou uma mensagem caso a lista esteja vazia.
- Na `main`, crie dois pedidos com bebidas diferentes, um `Entregue` e um `Cancelado`, e exiba o valor total de cada um.

nao precisa montar o codigo copleto nem nada disso,
primeiro separa por etapas o que eu preciso fazer e o
que utilizar pra cada etapa



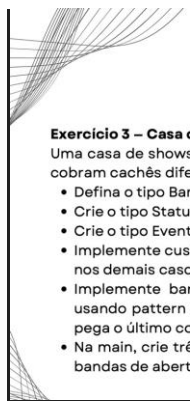
EXERCÍCIO 2

Exercício 2 – Lojas de Hyrule

Link está percorrendo as lojas de Hyrule e precisa calcular quanto vai gastar nos itens que quer comprar. Alguns itens custam mais caro e podem garantir um desconto no total.

- Defina o tipo Item com os campos nome, categoria e preco.
- Crie o tipo Compra com uma lista de itens e um StatusCompra com os construtores Pendente, Concluida e Cancelada.
- Implemente totalItens :: [Item] -> Double que soma os preços usando map e sum.
- Implemente valorFinal :: Compra -> Double usando guards: retorne 0.0 se Cancelada, aplique 10% de desconto se o total passar de 200, e retorne o total simples nos demais casos.
- Na main, crie uma compra com pelo menos três itens e exiba o valor final.

nao precisa fazer nadaaaa, so separa cada clisa que eu
tenho q fazer, em etapas



EXERCÍCIO 3

Exercício 3 – Casa de Shows

Uma casa de shows precisa calcular quanto vai custar cada evento da noite. Bandas diferentes cobram cachês diferentes, e o evento pode estar ativo, encerrado ou cancelado.

- Defina o tipo Banda com os campos nome, genero e cache.
- Crie o tipo StatusEvento com os construtores Ativo, Encerrado e Cancelado.
- Crie o tipo Evento com uma lista de bandas e um StatusEvento.
- Implemente custoTotalEvento :: Evento -> Double usando guards: retorne 0.0 se Cancelado, e nos demais casos some todos os cachês e adicione 20% de taxa de produção.
- Implemente bandaAbertura :: Evento -> String e bandaEncerramento :: Evento -> String usando pattern matching – a abertura pega o primeiro elemento da lista e o encerramento pega o último com last. Ambas devem retornar uma mensagem caso a lista esteja vazia.
- Na main, crie três eventos – um Ativo, um Encerrado e um Cancelado – e exiba o custo e as bandas de abertura e encerramento de cada um.

Exatamente. Esse é um dos motivos pelos quais esses
exercícios costumam vir em sequência.

O Exercício 3 é praticamente uma cópia da estrutura do
Exercício 2:

Exercício 2	Exercício 3
Item	Banda
Compra	Evento
StatusCompra	StatusEvento

preco	cache
totalItens	totalCache
s	s
valorFinal	custoTotal
al	Evento

A única diferença relevante é:

Exercício 2

Você só tinha:

```
valorFinal
```

Exercício 3

Além do custo, você precisa criar:

```
bandaAbertura
bandaEncerramento
```

Abertura é praticamente a mesma lógica que você fez em:

```
primeiraBebida
```

do Exercício 1.

Você já sabe fazer:

```
l
(x)
```

Já o encerramento é parecido, mas usando:

```
list
```

para pegar o último elemento da lista.

Minha sugestão:

1. Copie a estrutura do Exercício 2.

2. Renomeie os tipos (Item → Banda, Compra → Evento).
3. Faça primeiro o custoTotalEvento.
4. Depois faça bandaAbertura.
5. Por último faça bandaEncerramento.
6. Monte a main.

Quando terminar, me manda o código que eu faço uma revisão igual fizemos nos outros exercícios.

```
module Main where -- TIPOS data Banda = Banda {
nome :: String , genero :: String , cache :: Double } data
StatusEvento = Ativo | Encerrado | Cancelado data
Evento = Evento { bandas :: [Banda] , status ::
StatusEvento } -- FUNÇÕES custoTotalEvento ::
Evento -> Double custoTotalEvento e | Cancelado <-
status e = 0.0 | otherwise = sum (map cache (bandas e))
* 1.2 bandaAbertura :: Evento -> String bandaAbertura
(Evento [] _) = "Nenhuma banda no evento."
bandaAbertura (Evento (x:_) _) = "Banda de abertura: "
++ nome x bandaEncerramento :: Evento -> String
bandaEncerramento (Evento [] _) = "Nenhuma banda no
evento." bandaEncerramento (Evento bs _) = "Banda de
encerramento: " ++ nome
```

Está quase certo. O único ponto incompleto é essa última linha:

```
bandaEncerramento (Evento bs _) = "Banda de encerramento: "
++ nome
```

Você precisa aplicar nome na última banda da lista usando last:

```
bandaEncerramento :: Evento -> String
bandaEncerramento (Evento [] _) = "Nenhuma banda no
evento."
bandaEncerramento (Evento bs _) = "Banda de encerramento: "
++ nome (last bs)
```

O resto está correto.

bs representa a lista de bandas.

last bs pega a última banda da lista.

nome (last bs) pega o nome dessa última banda.

oq esta faltando pra ficar 100% igual foi solicitado no exercicio?

Comparando seu código com o enunciado, falta basicamente a main e um detalhe de organização.

Você já fez:

✓ Banda

✓ StatusEvento com:

Ativo | Encerrado | Cancelado

✓ Evento

✓ custoTotalEvento

✓ Retornar 0.0 se cancelado

✓ Somar os cachês

✓ Adicionar 20% de taxa

✓ bandaAbertura

✓ bandaEncerramento

✓ Pattern matching na lista vazia

✓ Uso de last

Falta 1 — A main

O enunciado pede:

"Na main, crie três eventos – um Ativo, um Encerrado e um Cancelado – e exiba o custo e as bandas de abertura e encerramento de cada um."

Então você precisa criar:

```
eventoAtivo  
eventoEncerrado  
eventoCancelado
```

E mostrar para cada um:

```
custoTotalEvento  
bandaAbertura  
bandaEncerramento
```

Falta 2 — (Opcional) Função auxiliar

O exercício não pede explicitamente, mas assim como no Ex. 2 você criou:

```
totalItem;
```

poderia criar:

```
totalCaches :: [Banda] => Double
```

e usar dentro de:

```
custoTotalEvento
```

Mas isso é apenas organização. Seu código atual já atende ao requisito.

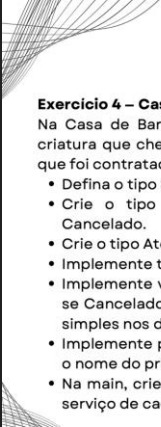
Então, para ficar 100% de acordo com o enunciado:

Falta apenas:

```
MAIN  
|— criar bandas  
|— criar evento Ativo  
|— criar evento Encerrado  
|— criar evento Cancelado  
|— mostrar custoTotalEvento
```

```
— mostrar bandaAbertura
— mostrar bandaEncerramento
```

Todo o restante da lógica já está conforme o que foi solicitado. ✓



EXERCÍCIO 4

Exercício 4 – Casa de Banhos da Yubaba

Na Casa de Banhos do mundo espiritual, Yubaba registra os serviços prestados a cada criatura que chega. Cada atendimento pode ter vários serviços, e o valor final depende do que foi contratado.

- Defina o tipo `Servico` com os campos `nome`, `tipo` e `preco`.
- Crie o tipo `StatusAtendimento` com os construtores `EmAndamento`, `Finalizado` e `Cancelado`.
- Crie o tipo `Atendimento` com uma lista de serviços e um `StatusAtendimento`.
- Implemente `totalServicos :: [Servico] -> Double` que soma os preços usando `map` e `sum`.
- Implemente `valorFinalAtendimento :: Atendimento -> Double` usando `guards`: retorne `0.0` se `Cancelado`, aplique `25%` de acréscimo se houver mais de `3` serviços, e retorne o total simples nos demais casos.
- Implemente `primeiroServico :: Atendimento -> String` usando `pattern matching` – retorne o nome do primeiro serviço ou uma mensagem caso a lista esteja vazia.
- Na `main`, crie dois atendimentos com serviços variados e exiba o valor final e o primeiro serviço de cada um.

me separa tudo que preciso realizar

Claro. Esse exercício pede estas etapas:

Etapa 1 — Criar o tipo Servico

Campos:

```
nome
tipo
preco
```

Usar:

```
data Servico = ..
```

Etapa 2 — Criar o tipo

StatusAtendimento

Construtores:

```
EmAndamento
Finalizado
Cancelado
```

Usar:

```
data StatusAtendimento = ..
```

Etapa 3 — Criar o tipo

Atendimento

Campos:

lista de serviços

status do atendimento

Usar:

data Atendimento = ..

Etapa 4 — Criar totalServicos

Assinatura:

totalServicos :: [Servico] -> Double

Objetivo:

- receber uma lista de serviços
- pegar o preço de cada serviço
- somar todos os preços

Usar:

map

sum

Etapa 5 — Criar

valorFinalAtendimento

Assinatura:

valorFinalAtendimento :: Atendimento -> Double

Regras:

1. Se estiver Cancelado, retorna 0.0
2. Se tiver mais de 3 serviços, aplica 25% de acréscimo
3. Caso contrário, retorna o total normal

Usar:

```
guards
```

Etapa 6 — Criar primeiroServico

Assinatura:

```
primeiroServico :: Atendimento -> String
```

Regras:

1. Se a lista estiver vazia, retorna uma mensagem
2. Se tiver serviços, retorna o nome do primeiro serviço

Usar:

```
pattern matching
```

com:

```
[]
```

```
(x:_)
```

Etapa 7 — Criar a main

Na main, você precisa:

1. Criar alguns serviços
2. Criar dois atendimentos com serviços variados
3. Exibir:
 - a. valor final de cada atendimento
 - b. primeiro serviço de cada atendimento

Resumo do que usar:

```
data
```

```
record syntax
```

```
lista [Service]
```

```
map
```

```
sum
```

```
guards
```

```
length
```

```
pattern matching
```

```
main
```

```
module Main where -- TIPOS data Servico = Servico {  
nome :: String , tipo :: String , preco :: Double } data  
StatusAtendimento = EmAndamento | Finalizado |  
Cancelado data Atendimento = Atendimento { servicos  
:: [Servico] , status :: StatusAtendimento } --  
FUNÇÕES totalServicos :: [Servico] -> Double  
totalServicos ss = sum (map preco ss)  
valorFinalAtendimento :: Atendimento -> Double  
valorFinalAtendimento a | Cancelado <- status a = 0.0 |  
length (servicos a) > 3 = total * 1.25 | otherwise = total  
where total = totalServicos (servicos a)
```

Até aqui está correto. ✔

Você já completou:

Etapa 1

```
data Servico = ..
```

Etapa 2

```
data StatusAtendimento = EmAndamento | Finalizado  
Cancelado
```

Etapa 3

```
data Atendimento = ..
```

Etapa 4

```
totalServicos :: [Servico] -> Double  
totalServicos ss = sum (map preco ss)
```

Usou exatamente o que o exercício pediu:

```
map
```

```
sum
```

Etapa 5

```
valorFinalAtendimento :: Atendimento -> Double
valorFinalAtendimento a
  | Cancelado << status a   = 0.0
  | length (servicos a) > 3 = total * 1.25
  | otherwise               = total
  where total = totalServicos (servicos a)
```

Também está correta.

A lógica ficou:

```
Cancelado -> 0.0

Mais de 3 serviços -> acréscimo de 25%

Demais casos -> valor normal
```

O que falta?

Apenas:

1. primeiroServico

Assinatura:

```
primeiroServico :: Atendimento -> String
```

Usando pattern matching:

```
|
| (C)
```

bem parecido com a sua primeiraBebida do Exercício 1.

2. A main

Criar:

- alguns serviços
- dois atendimentos

e exibir:

valorFinalAtendimento

primeiroService

para cada atendimento.

Então você já fez aproximadamente 80% do exercício.

O que falta é basicamente copiar a lógica da

primeiraBebida e depois montar a main.

consigo baixar em pdf toda a conversa desse chat pra

colocar no IA_REPORT do relatório?

Sim, mas o ChatGPT não tem um botão para eu gerar

automaticamente um PDF com **toda a conversa**.

Você tem algumas opções:

Opção 1 — Exportar a conversa inteira

No ChatGPT Web:

1. Clique na sua foto de perfil.
2. **Configurações.**
3. **Dados e controles.**
4. **Exportar dados.**
5. Você receberá um e-mail com um arquivo ZIP contendo suas conversas.

Opção 2 — Imprimir para PDF

No navegador:

1. Abra este chat.
2. Ctrl + P (Windows) ou Cmd + P (Mac).
3. Destino:
 - a. **Salvar como PDF**
4. Salve.

Essa costuma ser a forma mais rápida para anexar ao IA_REPORT.

Opção 3 — Gerar um relatório resumido

Para relatório acadêmico, normalmente o professor não quer centenas de mensagens.

Eu posso gerar um documento tipo:

```
IA_REPORT - Paradigmas de Programação

Objetivo do uso da IA
Como a IA auxiliou
Exercício 1
- discussão sobre tipos algébricos
- uso de guards
- uso de pattern matching

Exercício 2
- uso de map e sum
- cálculo de desconto

Exercício 3
- uso de last
- reutilização de estruturas

Exercício 4
- uso de length
- aplicação de acréscimo por quantidade de serviços

Aprendizados obtidos
Dificuldades encontradas
```

que fica muito mais profissional para entregar.

Se seu professor pediu um IA_REPORT, eu recomendo essa terceira opção. Posso montar um relatório completo baseado em tudo que fizemos neste chat.

